

TORRE EV

Painel do Síndico — Gestão de Recarga de Veículos Elétricos

Documentação Técnica do Projeto

Estudo de Caso & Desafio Prático
Programação de Dispositivos Móveis
Plataforma: MIT App Inventor

Integrantes: _____

Turma: _____ Data: ____/____/____

Sumário

1. Definição do problema — dor resolvida e usuário final
2. Visão geral da solução — funcionalidades e requisitos técnicos atendidos
3. Telas da solução — estrutura, visual e navegação do aplicativo
4. Diagrama de blocos — lógica do algoritmo no App Inventor
5. Proposta de valor — por que a solução é eficiente para um condomínio antigo
6. Roteiro de construção — passo a passo no MIT App Inventor

1. Definição do problema

Contexto. A Residencial Parque das Torres é um condomínio antigo (construído no início dos anos 2000) com 120 apartamentos e 240 vagas de garagem. A rede elétrica foi dimensionada apenas para iluminação, portões e elevadores. Com a chegada de veículos elétricos (15 hoje, tendência de dobrar), a tentativa de recarga em tomadas comuns causou quedas de disjuntores, aumento da conta de energia e conflitos entre vizinhos.

Dor específica escolhida (nicho): a incapacidade do síndico e da administração de monitorar e controlar as recargas na garagem comum. São três dores concretas dessa figura do condomínio:

- Risco de sobrecarga na infraestrutura: medo de exceder o barramento principal, desligando elevadores e danificando equipamentos.
- Ausência de controle de uso: ninguém sabe quanto cada morador consome, nem quando há recargas simultâneas demais.
- Falta de histórico para cobrança: sem registro de kWh por morador, não há como cobrar com justiça pelo que cada um usou.

Usuário final: o síndico do condomínio, que precisa de um painel simples de consulta e alerta no celular.

2. Visão geral da solução

O aplicativo Torre EV é um painel de controle para o síndico acompanhar, em tempo real, a situação da recarga de veículos elétricos na garagem comum. Ele soma a potência (kW) das recargas ativas, compara com o limite elétrico do condomínio e emite alertas visuais e sonoros apenas quando o status muda. O painel é atualizado automaticamente a cada 2 segundos (Clock1.Timer), sempre com os dados mais recentes do TinyDB. Todo o cadastro e o histórico ficam armazenados no TinyDB, não se perdendo ao fechar o app.

Principais funcionalidades:

- Login simples do síndico (senha padrão 1234, alterável via TinyDB) para acesso restrito ao painel.
- Cadastro de recarga: nome do morador e potência do carregador (kW), com validação de dados e de capacidade.
- Dashboard: soma automática de kW em uso, limite padrão de 22 kW, barra de carga e cartão de status com semáforo (verde/amarelo/vermelho); atualização automática a cada 2 segundos.
- Alertas por transição de status: notificação e voz somente quando o status muda (evita notificações repetitivas a cada segundo).
- Encerramento de recarga: cálculo de kWh (potência x duração) e registro no histórico com data/hora.
- Histórico de recargas em ListView, com nome do morador, kWh, duração e data.

Requisitos técnicos atendidos (edital): condicional encadeada (se...senão), estruturas de repetição (para cada), procedimentos com retorno (função CalcularTotal), persistência local (TinyDB), armazenamento e manipulação de listas, componentes multimídia (TextoParaFala), sensores de data/hora (Clock) e design responsivo para celular.

3. Telas da solução

O aplicativo é organizado em 4 telas, garantindo navegação fluida e coerente. O visual usa cartões brancos sobre fundo cinza-claro, títulos e botões em azul-escuro (paleta #0D47A1/#002171) e destaque vermelho para situações de emergência.

Tela	Função	Componentes principais
1. Screen1 (Login)	Protege o acesso ao painel. O síndico digita a senha; se correta, abre o dashboard.	CardLogin, TextBox1 (senha, Password), Button1 Entrar, Label1 (erro), Notifier1
2. Screen2 (Painel)	Tela central: cartão de status (verde/amarelo/vermelho), barra de carga, total em kW, limite, vagas e botões de ação.	CardStatus, CardUso, CardAcoes, Slider1 (barra de carga, desabilitado), ListPicker1 (recargas ativas), Buttons Nova Recarga/Encerrar/Histórico, Notifier1, TextToSpeech1, Clock1
3. Screen3 (Cadastro)	Registra nova recarga com validação de dados e de capacidade.	CardCad, TextBox1 (nome), TextBox2 (kW, aceita ponto ou vírgula), Button1 Salvar, Button2 Voltar, Notifier1
4. Screen4 (Histórico)	Lista o histórico de recargas encerradas.	CardHist, ListView1, Button1 Voltar

Navegação: Screen1 > Screen2 (login) · Screen2 > Screen3 (nova recarga) > volta · Screen2 > Screen4 (histórico) > volta.

4. Diagrama de blocos

Os blocos abaixo descrevem o algoritmo implementado em cada tela, com destaque para a lógica condicional encadeada (se...então...senão), o procedimento com retorno e o uso do TinyDB. Os prints das telas de código devem ser anexados a esta seção após a construção do protótipo.

4.1 Screen1 — verificação de senha

```
when Screen1.Initialize
call TinyDB1.GetValue tag "senha" valueIfTagNotThere "1234"
when Button1.Click
if TextBox1.Text = senha
then: open another screen "Screen2"
else: Notifier1.ShowAlert "Senha incorreta"
```

4.2 Screen2 — monitoramento com alerta por transição de status

```
when Screen2.Initialize
global limite = TinyDB1.GetValue "limiteKw" valueIfTagNotThere 22
global rec = TinyDB1.GetValue "recargasAtivas" (lista vazia se nao houver)
call AtualizarPainel
when Clock1.Timer (a cada 2 segundos)
call AtualizarPainel -> painel sempre atualizado
procedure AtualizarPainel
total = soma dos valores kW das recargas ativas (funcao CalcularTotal)
Label4.Text = total + " kW em uso de " + limite + " kW"
Label6.Text = "Limite: " + limite + " kW"
Label7.Text = "Vagas ocupadas: " + length(rec)
Slider1.ThumbPosition = total
se nomes das recargas ativas mudou (nomes != nomesAnt)
entao: atualiza ListPicker.Elements e mantem a selecao atual
se total > limite (sobrecarga)
then: se statusAnt diferente de "sobrecarga"
CardStatus vermelho, Label2 "SOBRECARGA!", Notifier1.ShowAlert, TextToSpeech1.Speak
else if total >= 0.8 x limite (atencao)
then: se statusAnt diferente de "atencao"
CardStatus amarelo, Label2 "ATENCAO", Notifier1.ShowAlert, TextToSpeech1.Speak
else (ok)
then: se statusAnt diferente de "ok"
CardStatus verde, Label2 "OK", Notifier1.ShowAlert "Carga normal"
statusAnt recebe o novo status (so alerta na mudanca de estado)
```

Nova recarga: o botão abre a ListPicker com as recargas ativas; escolher uma e Encerrar remove da lista, calcula kWh = kW x duração (Clock1), guarda no TinyDB tag "historico" e atualiza o painel.

4. Diagrama de blocos (continuação)

4.3 Screen3 — cadastro com validação encadeada

```
when Screen3.Initialize
global limite = TinyDB1.GetValue "limiteKw" valueIfTagNotThere 22
global recargas = TinyDB1.GetValue "recargasAtivas"
when Button1.Click (Salvar)
if TextBox1.Text = ""
then: Notifier1.ShowAlert "Informe o nome do morador"
else if TextBox2.Text = "" ou NÃO(EhNumero(TextBox2.Text)) ou potência < 1 ou potência > limite
then: Notifier1.ShowAlert "Potencia invalida (1 a " + limite + " kw)"
else if (CalcularTotal(recargas) + potência) > limite
then: Notifier1.ShowAlert "Nao permitido: excederia o limite"
else
adiciona [nome, potência, duracao, inicio] em "recargasAtivas" (TinyDB)
Notifier1.ShowAlert "Recarga cadastrada"
open another screen "Screen2"
procedure EhNumero(texto)
retorna (é número? texto) OU (é número? substitui "," por "." em texto)
-> aceita "7" e "7,2" como potências válidas
procedure CalcularTotal(lista)
retorna a soma das potencias (2o item de cada sublista)
```

4.4 Screen4 — exibição do histórico

```
when Screen4.Initialize
global historico = TinyDB1.GetValue "historico"
for each item em historico:
monta linha "Morador: " + item(1) + " | " + item(2) + " kWh | "
+ item(3) + " h | " + item(4)
adiciona em exibicao
ListView1.Elements = exibicao
```

5. Proposta de valor

- Zero obra na rede elétrica: o condomínio de 20 anos não precisa de retrofit imediato no barramento. O app gerencia a demanda em kW e impede que as recargas ultrapassem o limite de 22 kW, reduzindo o risco de incêndio e de queda dos disjuntores.
- Prevenção de desastres e de conflitos: o alerta sonoro e visual de sobrecarga avisa o síndico em tempo real, permitindo ação imediata antes que os elevadores parem ou um cabo sobrecarregue o barramento.
- Transparência para todos os atores: com o histórico de kWh por morador registrado no TinyDB, o síndico consegue cobrar exatamente o que cada um consumiu — atendendo o desejo de "pagar pelo que usei".
- Usabilidade: interface com cartões, semáforo de status e alertas apenas na mudança de estado — simples o bastante para o síndico usar no dia a dia sem treinamento.

Semáforo de status (regra de negócio):

- Verde: total em uso menor que 80% do limite (22 kW).
- Amarelo: total em uso entre 80% e 100% do limite — atenção.
- Vermelho: total em uso acima do limite — sobrecarga (alerta sonoro).

6. Roteiro de construção no MIT App Inventor

Passo a passo para montar o protótipo (ou importar o TorreEV.aia em "Importar projeto (.aia)"):

- 1. Criar o projeto e as 4 telas (Screen1 a Screen4); em "Propriedades da Tela", configurar Title e Screen1 como tela inicial.
- 2. Na Screen1 (Login): TextBox1 (senha, Password marcado), Button1 Entrar, Label1 de erro e Notifier1. Implementar os blocos de 4.1.
- 3. Na Screen2 (Painel): VerticalArrangements para os cartões CardStatus/CardUso/CardAcoes (BackgroundColor branco), Slider1 como barra de carga (Enabled desmarcado), ListPicker1, botões, Notifier1, TextToSpeech1, Clock1 e TinyDB1. Implementar 4.2.
- 4. Na Screen3 (Cadastro): TextBox1 (nome), TextBox2 (kW, aceita ponto ou vírgula), Button1 Salvar, Button2 Voltar e Notifier1. Implementar 4.3.
- 5. Na Screen4 (Histórico): ListView1, Button1 Voltar e carregar o histórico na Initialize. Implementar 4.4.
- 6. Conectar o celular com o app AI Companion e testar o fluxo completo: login, cadastro, sobrecarga simulada e histórico.

Dica: para testar o alerta de sobrecarga, cadastre recargas cuja soma ultrapasse 22 kW ou reduza temporariamente o valor em "limiteKw" no TinyDB.